Agentic CV–JD Ranking for Automation Test Roles: Final Report

NLP501 · Group 2 · Sơn (problem and criteria) · Vũ (literature) · Đán (agentic HR, gaps, the 8-axis rubric and the `coding` set) · Hoa (datasets and metrics) · Thịnh (architecture and experiments)

The official numbers come from **NLP501-bench v1**, the benchmark the team agreed on and froze (`docs/BENCHMARK.md`). Every current number is generated by a command in §11 and can be checked in the `docs/*_RESULTS*.md` file named next to it. Numbers from earlier runs, kept to explain a design change, are marked as such: v2, qwen3:8b, before a fix, or an uncached run.

Abstract

We built an agent that ranks Automation Test Engineer CVs against a job description and explains every rank. We evaluated it against the three questions that matter most for an agentic NLP system:

1. Is it genuinely agentic?
2. Is the data processed well?
3. How is what the LLM produces controlled?

The team froze one benchmark, NLP501-bench v1: 250 synthetic, anonymised CVs and 18 JDs, a locked test split of 10, and every input pinned by checksum. On that test split:

- **The final system is the best version of the project:** 0.851 nDCG@10 for the full agent and 0.839 for its fixed pipeline. That is ahead of the rubric-only ranker it grew from (0.810), our v2 (0.748, $p = 0.009$ for the deterministic system) and every retrieval baseline (0.68–0.72). Its lead over the rubric ranker is not yet significant ($p = 0.12$–0.24).
- **Rules score; the LLM controls and explains.** The agent and its fixed twin rank almost alike (0.851 vs 0.839, +0.012, $p = 0.064$, not significant), and most of the gap is the one vague test JD, where the agent asks HR. The agent's value is the work rules cannot do: asking HR on a vague JD, reading CVs the parser cannot, catching contradicted claims (2.3% → 1.5%) and keeping the arbiter to true near-ties.
- **Where the agent decides, the ranking shows it.** On vague twins of all 18 JDs, a pipeline that cannot ask scores 0.626 nDCG@10. Asking HR recovers 57% of the loss (+0.129, $p < 0.001$); the agent as it runs scores 0.704 (§5.5).
- **Evaluation found and fixed real faults:**
  - a recall funnel sized for 50 CVs that failed at 200 (the agent rose from 0.60 to 0.77–0.78 on those JDs);
  - an arbiter acting beyond its mandate;
  - a 100% name leak in masking;
  - nine more defects in data processing, three of them found by reading scanned CVs through OCR.
- **A stronger model needed more control, not less.** When Claude instead of qwen3:8b read the JDs, the LLM-only JD parser fell from 0.884 to 0.842 nDCG@10 on our JDs. The rules-first hybrid parser did not move.

- **v4: an LLM orchestrator over the same tools (P7).** The model chooses the next of 17 typed tools; a code harness checks every call. Live on HR tasks (E15) it passed **40 of 42 behaviour checks** and exposed one real fault (an HR rejection did not take the CV out of the ranking), now fixed. On complex HR goals (E26, three runs per goal, cache off) it passed **81 of 83**: it turns the goal into constraints that quote HR, checks the pool can meet them, and goes back to HR when it cannot (§3.1, §5.7).
- **Real CVs (NLP501-bench v2).** Eight real QA Automation JDs from Djinni were added to the test split (18 test JDs). The ladder holds on real text: BM25 0.441 → fixed pipeline 0.723 on Djinni; 0.582 → 0.787 over all 18 (§5.8). On Djinni with human labels (424 pairs, two graders each), BM25 0.422 → 8-axis rules 0.636 → P6 full agent **0.643** (+0.221 over B0, $p < 0.001$, P@5 = 0.725) (§2.3).
- **Every LLM output passes code checks at run time** (Q1–Q7 and Q14–Q20 of 20 written criteria; the rest are offline evaluations).
- **Human labels.** Two teammates graded 120 pairs of `ours` blind (κ 0.965 with each other). Against the silver mean κ is 0.66–0.70, so silver is the secondary set under the registered rule. Re-scoring v1 with the human pairs (v1.1) leaves the order of the eight systems unchanged (Kendall τ 1.00 on test and dev): comparisons between systems hold (§9).
- **The ranking ignores the protected attributes we twinned** (name, gender, age, marital status): no score changed in 6 twin variants × 1,800 (CV, JD) pairs.

1. Introduction

The problem

The task. An HR recruiter hiring an Automation Test Engineer has a job description (JD), a pool of CVs and, more and more often, a goal in their own words: *"5 CVs, Selenium a must, Playwright nice but other criteria are fine, AI a must, AI used in automation a big plus."* They need:

- a ranked shortlist that follows that goal and their company type's criteria;
- for every recommended CV, a reason quoted from the CV that they can check;
- for every other CV, a status and a reason (knocked out, not shortlisted, needs review);
- when the goal cannot be met on this pool, to be told so before anything is ranked, with what relaxing it would give.

Input	a JD (English or Vietnamese, often vague); HR's goal and later turns ("why is #1 above #2?", "make Docker a must", "reject CV-014"); a CV pool (PDF, DOCX, scans; 50–250 CVs)
Output	the top N, each with per-criterion scores, quotes verified against the CV, gaps and interview questions; a status and reason for every other CV; the parts of HR's goal that could not be met, stated
HR's role	owns every decision: answers questions, approves tool substitutes, re-weights, approves and rejects

Why it is hard. Keyword ATS cannot see three things HR must judge:

- **Depth:** does the candidate run scripts (L1), write them (L2), build a framework (L3) or design test architecture (L4)?
- **Use vs mention:** was a tool used, or only listed?

- **Hedging:** is a mention hedged ("familiar with", "currently learning")?

HR's goals add two more. They mix hard and soft constraints with concepts no skill list holds ("AI in automation"), and some cannot be met on the pool at hand (Cypress *and* Appium: 1 CV of the 5 wanted).

What success means. Ranking quality on a frozen benchmark (§5), and three questions about an agentic NLP system:

1. **Is it agentic?** It plans per goal, chooses its tools, asks HR when the answer would change the result, checks its own output against the goal and states what it could not deliver (§3, §5.7).
2. **Is the data processed well?** Extraction, masking, OCR and real text, each measured (§2).
3. **Is what the LLM produces controlled?** 20 written criteria, each with a code check and an action on failure (§4).

Constraints. No real CV without consent; protected attributes are deleted before indexing; rules set every score, never the LLM; HR decides.

Our approach

Keyword matching rewards stuffing. An LLM judge is opaque, position-biased and can invent evidence. We rank with

`Score(CV, JD) = 1[H(c)] × Σ_k w(c, k) · S(c, k)`

where:

- H is a hard knock-out on must-have tools;
- w are per-company-type weights over 8 criteria;
- S are 0–100 criterion scores.

Contributions:

1. A LangGraph agent with 12 nodes and 6 bounded loops:
    - rules set every score;
    - the LLM explains and decides only what rules cannot;
    - humans own the decision.
2. A deterministic, bilingual CV/JD NLP pipeline, measured against another team member's generator truth, real text and counterfactual twins.
3. 13 written criteria for AI output (Q1–Q13), each with a check and a target. Q1–Q7 run on every output at run time, with an action on failure; Q8–Q13 are offline evaluations.
4. **NLP501-bench v1**, the team's frozen benchmark, with a head-to-head of the three versions of the project.
5. A goal-driven orchestrator (P7): HR's goal becomes a typed spec that quotes HR, is checked against the pool before ranking, and whose unmet parts the answer must declare (§3.1, §5.7).

Research question	Answered in
RQ1 Does structured extraction beat raw-text matching?	§5.1
RQ2 Do HR-configured must-haves and weights fit the company better?	§5.4
RQ3 How much does hybrid retrieval + reranking add over keyword or dense alone?	§5.1
RQ4 What does agentic orchestration add over a fixed pipeline with the same tools?	§3, §5.2, §6
RQ5 Do results and errors differ by JD type and ambiguity?	§5.1, §8

1.1 Related work, and what our measurements say about it

Vũ's literature review (31 papers, in the midterm deck) shaped the design. Each strand below states what we took and what our results add.

- **Dense matching of CVs and jobs.** ConFit v2 (Yu et al., 2025) trains contrastive bi-encoders for resume–job matching. On test-automation CVs, dense retrieval was the weakest of the three whole-CV baselines on both datasets (all-JD nDCG@10 0.741 / 0.333, against BM25's 0.794 / 0.542). That confirms Resume2Vec (2025), which found that keyword ATS beats embeddings for software testing, where exact tool names decide relevance. Hence the mandatory hybrid retrieval and the cross-encoder.
- **LLM rerankers.** RankGPT (2023) and later listwise LLM rerankers rank with the model itself. We kept the LLM out of the score: the v2 measurement showed it cost 0.033 nDCG@10, and §7 (H2) shows that "LLM only where needed" is already what a well-built fixed pipeline does.
- **Skill normalisation.** Clavié & Soulié (2023) use LLMs as zero-shot ESCO skill matchers. We use a curated gazetteer with canonicalisation at the boundary, and found that the stronger the LLM, the more it over-reads JD requirements (§4). The deterministic parser had to stay in charge.
- **Bias in LM screening.** Wilson & Caliskan (2024) found that LM-based resume retrieval favoured White-associated names in 85.1% of tests and female-associated names in only 11.1%, and NYC Local Law 144 requires bias audits of automated hiring tools. Our response is deletion before indexing, audited by counterfactual twins: no score moved.
- **Agent patterns.** These map onto loops in our graph:
  - ReAct (Yao et al., ICLR 2023), reasoning interleaved with tool calls, is the investigator's loop (§3.1).
  - Reflexion (Shinn et al., NeurIPS 2023), self-critique and retry, is L4, except that the critique comes from code, not from the model.
  - AutoGen (Wu et al., 2023) coordinates several agents. We chose one graph with bounded loops, because every decision point stays testable and measurable.

2. Data, processing and the benchmark

2.1 Data

	ours-silver (`data/`)	coding-rubric (template set)	real text
Size	50 CVs × 12 JDs	200 CVs × 6 JDs	parsing: 217 CVs; Djinni-QA: 250 CVs × 8 JDs, 424 pooled pairs
Built by	hand-written synthetic, designed to stress the pipeline (Vietnamese, long, hedges, title inflation)	template generator with known latent facts; 37% Vietnamese; 7 OCR-garbled CVs; fake PII	datasetmaster/resumes and Djinni public datasets (MIT); synthetic/Faker rows excluded
Labels (0–3)	silver plus a blind 120-pair Hoa–Phúc audit	rubric gold (`gold_spec`)	Djinni: Vũ–Đán Aggregate Human Gold; parsing set: none

The team does not commit real CV text. Datasetmaster and Djinni are used under their public licences; Djinni's authors anonymised the profiles. Runtime text stays under `.runtime/`. Any CV collected directly by the team still requires the consent process in `docs/labeling/CONSENT.md`.

2.2 NLP501-bench v1 (`docs/BENCHMARK.md`)

The two datasets were built in parallel, and each defined "relevant" its own way, so each system looked best at home. The team therefore agreed on one benchmark with fixed roles:

	dev (tuning allowed)	test (locked; the official number)
ours	JD-001, 002, 003, 006, 008, 012	JD-004, 005, 007, 009, 010, 011
coding	ambiguous_junior, mobile_offshore	ambiguous_senior, bank_api_qa, fpt_automation_test, startup_framework

How it works: - Each JD ranks the CVs labelled for it. - Our JDs were split by company type with seed 501; the coding JDs keep the split their author pre-registered. - A manifest pins 268 input files by SHA-256, and a test fails if any of them changes. - Changes to the system are decided on dev and reported on test.

Disclosure: results on these JDs were seen during development, before the split existed. Of the two fixes in §5.3, the recall channel was motivated by a test-split error and decided on dev. The arbiter fix restores the specified behaviour and was found in a dev-JD trace. Both are re-reported on test.

2.3 Human validation and real-data external pilot

Hoa and Phúc independently graded a blind sample containing all 94 Claude A/B disagreements plus 26 agreeing controls. Human–human agreement is 90.8%, κ = 0.965. Against the registered half-up-rounded silver mean, κ is 0.695 and 0.661, so the silver benchmark is secondary rather than a replacement for people (`HUMAN_LABELS.md`).

For external validity, Vũ and Đán graded the same 424 pooled Djinni pairs (Đán's set with the model's grade visible). Aggregate Human Gold keeps their unrounded mean and both source grades: exact agreement 67.0%, quadratic-weighted κ = 0.696; 284 labels have the same grade, 111 differ by one and 29 differ by at least two. In the frozen production run, B0/B1/B2/B3 score 0.422/0.464/0.482/**0.636** nDCG@10 and the full agent P6 **0.643 [0.563, 0.725]** (+0.221 over B0, $p < 0.001$; conservative P@5 of **0.725 [0.575, 0.875]**, τ-b of **0.415**; `DJINNI_PRODUCTION_RESULTS.md`). P6 ran on Gemini 3.8 Flash in this run; rules set its score, as everywhere else.

A fixed-seed holdout of 6 new JDs and 40 new CVs, never seen during development, is locked by source and content hashes before any run. Code refuses to materialize or score it until two independent blind human files cover all 240 pairs and pass the registered κ/adjudication gate. It is not yet a result (`DJINNI_HOLDOUT.md`).

2.4 Pipeline: raw CV → facts (deterministic, 0 tokens)

The pipeline runs these steps in order:

1. **Normalise:** NFC, bullets, wrapped lines.
2. **Mask** contact details and protected attributes.
3. **Detect** sections and dated roles.
4. **Chunk structurally:** one chunk per role or project, labelled with its section.
5. **Find skill mentions** with the gazetteer, with NegEx-style hedge scope and evidence strength (used / listed / hedged).
6. **Compute years** from merged date ranges.
7. **Estimate automation depth** from the candidate's own action verbs.
8. **Output the `CVProfile`.**

JDs are parsed into must-haves with *one-of* groups, nice-to-haves, years, depth, languages and domains. Every fact keeps the CV line it came from (`docs/NLP_PIPELINE.md`).

Masking is deletion, not instruction. Asking an LLM to "ignore gender" is a hope; deleting it before indexing is a guarantee.

2.5 How well the processing works

Check (source)	Result
Tool extraction vs the coding generator's truth, 200 CVs (`EXTRACTION_RESULTS.md`)	F1 0.978; clean CVs 0.994; OCR-garbled CVs 0.192
Years of experience · automation depth	MAE 0.79 years · 89% exact
Redaction leak, every PII field	0%
Real CVs (217): parsed · prose precision (`REALTEXT_RESULTS.md`)	217/217 · 90%
Counterfactual twins: name / gender / age / marital status (`ROBUSTNESS_RESULTS.md`)	0 score changes in 6 twin variants × 1,800 (CV, JD) pairs
Quality flags on clean CVs	17/193 flagged, all for having no dated roles
Scanned CVs: 30 held-out CVs as scans and phone photos, read by PaddleOCR (`OCR_RESULTS_heldout.md`)	tool F1, years and sections identical to reading the text; 0% PII leak; CER 0.3–0.4% English, 5.7–6.1% Vietnamese

The extraction numbers were measured after fixing defects that this same measurement exposed, so they are optimistic for this set. The real-text and twin checks are independent of those fixes.

What measuring found and we fixed. Unit tests had passed on all of these:

1. **The candidate's name leaked in 100% of CVs whose first line is the bare name.** It now leaks in 0%.
2. **A gender value under an OCR-garbled label leaked.**
3. **The garbled-text flag caught 0 of 7 garbled CVs.** It now catches all 7.
4. **Double-spaced CVs lost their role titles,** which inflated one candidate's automation years from 8.0 to 14.8.
5. **"pull requests" counted as the *requests* library,** giving false credit to our own trap CV.

6. **Jira counted as automation work.**
7. **Tool names missing from the gazetteer,** found in the coding set and in real CVs.

Scanned CVs (OCR). The benchmark's "OCR-garbled" CVs are synthetic noise; no benchmark CV is a scan. Real candidates do send scans and photos, so `read_file` now reads a PDF page without a text layer, or a photo, with PaddleOCR (PP-OCRv5, an optional dependency). PaddleOCR has no Vietnamese model: its latin model drops letters with stacked diacritics ("KINH NGHIỆM" → "KINH NGHIM"). On 30 coding CVs rendered as scans, the first run found three more defects, all in how our pipeline read OCR output:

- **Capital I read as "l"** ("WebdriverlO", "GitLab Cl") caused every missed tool (recall 0.93).
- **Vietnamese headings with dropped letters** were not recognised: 0/15 Vietnamese CVs had an experience section.
- **PII labels with dropped letters** ("Đa chi:", "Giới tính:") were not masked: the address leaked on 15/15 Vietnamese CVs and the gender on 6/15.

The fixes: tool-name-only I/l correction, and headings and labels matched after folding diacritics. The diacritics fix also covers real CVs typed without accents ("Dia chi:"). After the fixes, the 30 development CVs read as well as their text, and **30 held-out CVs, never looked at, match their text exactly with 0 leaks** (`OCR_RESULTS_heldout.md`). Applied to the benchmark, the fixes changed only the 4 garbled CVs (two more sections found, partial PII such as "Ge@der: @em%l@" now masked). nDCG@10 is unchanged on all 18 JDs. The scans are rendered single-column CVs; two-column layouts and tables are untested.

Chunking (RQ1). With the cross-encoder, structural chunks are slightly best on both sets (0.829 / 0.628 vs whole CV 0.818 / 0.620, not significant). Without it, whole CVs retrieve best. The large gain is elsewhere: **rule scoring over the parsed profile adds +0.052 / +0.164 nDCG@10 over the same retrieval** (p ≈ 0.001 on ours; positive on all 6 coding JDs; `EVAL_RESULTS*.md`, all JDs).

3. Architecture: where the agent decides

The system has two layers (`docs/architecture.html`, `docs/agentic-architecture-workflow.md`). Their shared rule: **rules where facts exist, the LLM where judgement or reading is needed, humans where the decision is theirs.**

3.1 P7: an LLM orchestrator inside a code harness (v4, the API default)

`agent/orchestrator.py`. HR states a goal in their own words; the model then picks the next step from **17 typed tools** (`agent/tools.py`): `parse_jd`, `formalize_goal`, `define_concept`, `check_goal`, `ask_hr`, `screen`, `investigate`, `propose_substitutes`, `explain`, `rank_shortlist` (off by default, §9), `compare`, `get_candidate`, `search_cvs`, `update_requirements`, `set_weights`, `record_decisions`, `respond_to_hr`. `screen` is the P6 pipeline up to the ranking at 0 tokens, so the tools reuse the nodes below. It is a multi-turn conversation: HR can ask why, change a requirement, re-weight or reject a candidate, and the agent acts.

Goal-driven control (`agent/goal.py`). A complex goal such as "5 CVs, Selenium a must, Playwright nice, AI a must, AI in automation a big plus" becomes a typed plan. Code checks that every constraint quotes HR (Q19); a concept no taxonomy holds ("AI in automation") is defined once and graded by code; `check_goal` counts what the pool can deliver before anything is ranked; when the goal cannot be met the agent must go back to HR, and its answer must state what it could not deliver (Q20).

A **guard** (the harness) runs before every tool and refuses with a reason the model reads (Q14):

- schema-valid arguments and a known tool; each tool's precondition (`check_*`: no ranking before a screen, no recommendation before `explain` has verified the evidence);
- **HR authority:** requirements, weights and decisions change only if HR named the change in their own words, never because a CV or a tool result says so (Q16 provenance);
- what reaches HR is checked like any LLM output (Q15): real ids, verified candidates, supported claims, no protected attributes;
- budgets: `max_agent_steps`, `max_agent_tokens`, `max_hr_turns`; on a budget stop, code answers HR.

The score stays with the rules: no tool can set S or a knock-out. P6 below is kept as the fixed baseline.

3.2 P6: the fixed graph and its loops

P6 is a LangGraph state machine with 12 nodes and 6 bounded loops (`agent/graph.py`).

Loop / node	Decision	Decided by
L1 clarify	pause and ask HR, or proceed	rules first (no must-have tools; title contradicts years), then a typed LLM gate
L2 expand	widen the search; which related tools may substitute	coverage rule proposes, **HR approves** each tool
L6 investigate	which tool to call next on an unreadable CV, and what it shows	**LLM** (typed actions); **code** verifies every claim
L4 verify	retry a review whose quotes or claims fail the checks	code (Q2–Q5)
L3 arbiter	swap a *near-tie* (	Δ
L5 HR review	approve, re-weight, or approve/reject candidates	**HR**; weights re-learned from decisions (pairwise logistic regression, audit log); a rejected CV leaves the ranking (`rejected_by_hr`, since v4: E15 found that a reject only re-fitted the weights)

Why this is agentic rather than a linear pipeline. In P7 the model plans the order of work itself (§5.7). In P6 the path still depends on what the agent observes: across the 18 benchmark JDs it took 8 distinct paths; its fixed twin took 1. The investigator (N12) is a tool-using ReAct loop in the strict sense: the model chooses its next action from what the previous one returned. Code then decides whether the result becomes a fact.

Why the LLM does not score. In v2 the LLM scored 5 of 8 criteria, and the agent then ranked 0.033 below its own deterministic input. Since v3:

- rules set all 8 scores;
- the LLM's judgement is a verified review;
- the agentic parts do what rules cannot: ask, read noisy text, break ties, explain.

4. Controlling what the LLM produces

`docs/AI_OUTPUT_CRITERIA.md` defines 20 criteria (Q1–Q13 for v3, Q14–Q20 added with P7). Each has a code check, a target and an action on failure.

	Criterion	Check	Action on failure
Q1	schema-valid output	pydantic	retry, then safe default / needs review
Q2	quotes grounded in the CV	fuzzy span match	L4 retry naming the failing quotes
Q3	quotes support their criterion	right section and entity	labelled "listed, not demonstrated"
Q4	rationales consistent with the parsed CV	no tool or years the CV does not show	L4 retry with the contradiction, then `[unverified]`
Q5	no protected attributes	EN/VI lexicon	sentence removed
Q6	report claims supported per candidate	Q4 + Q5 per candidate, no foreign ids	regenerate once, then drop failing sentences
Q7	arbiter not position-biased, near-ties only	both orders;	Δ
Q8	decisions right	planted scenarios	safe default
Q9	explanation adds up to the score	by construction	—
Q10	ranking agrees with people	benchmark metrics with CIs	not adopted
Q11	ranking ignores protected attributes	counterfactual twins	release blocker
Q12	bounded cost	calls, tokens, latency	loop limits
Q13	an LLM judge is validated before use	κ vs humans	not used as ground truth
Q14	orchestrator calls valid and allowed	known tool, schema, per-tool precondition	refused with the reason; budget stop → code answers
Q15	what the agent tells HR is faithful	ranked and explained ids only; per-CV Q4; Q5	refused with the failing claims
Q16	untrusted text cannot steer the system	injection lexicon (+ optional decision model); HR provenance of every change	sentence removed and CV flagged; change refused
Q17	LLM ranking read, unbiased, bounded	shortlisted ids only; two passes in opposite orders	repaired at the boundary; agreement reported
Q18	UI language never changes a result	requirement extraction never localized; translations keep numbers and tools	English original shown
Q19	a formalized goal is HR's goal	every constraint quotes HR's words	refused; the model re-formalizes
Q20	the answer states what the goal could not get	shortfall vs the feasibility count	refused until declared

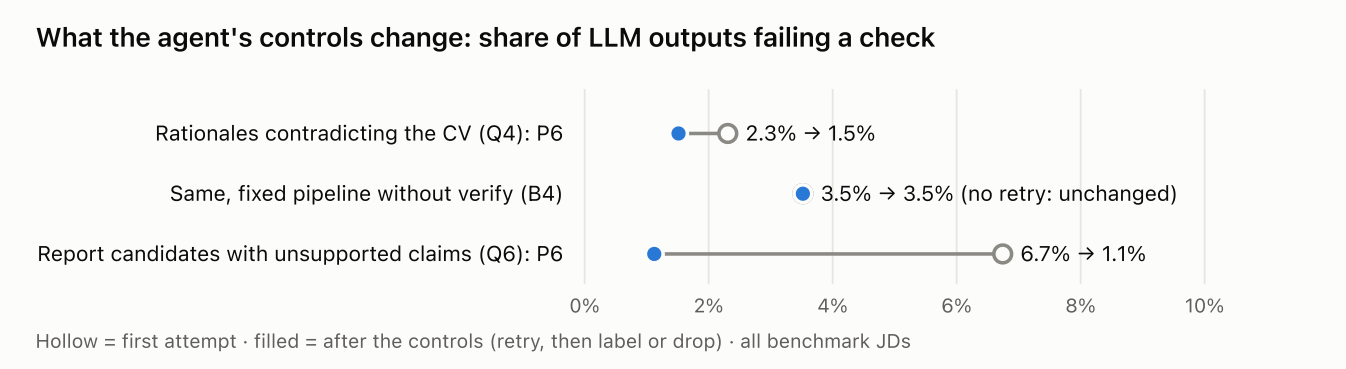


Figure 1. The share of LLM outputs failing a check, on the first attempt (hollow) and after the graph's controls acted (filled), across all benchmark JDs. Numbers: the AI-output scorecards in `EVAL_RESULTS*.md`.

Measured on the full agent across the benchmark JDs: - **Q1:** 0 invalid outputs. - **Q2:** 100% of quotes grounded. - **Q4:** contradicted rationales 2.3% → 1.5%, against 3.5% for the fixed pipeline, which has no retry. - **Q6:** report candidates with an unsupported claim 6.7% → 1.1% after one regeneration; those sentences dropped, none reaching HR. - **Q7:** after the |Δ| fix (§5.3), 17 of 17 benchmark near-tie pairs gave the same verdict in both orders. The JD-003 trace in §6.1 shows the check catching a flip. Before the fix, 90% were stable, when out-of-mandate pairs were also judged. - **Q8 decisions** (`SCENARIO_RESULTS*.md`): - **L1 gate:** in the Claude benchmark runs the agent asked HR on exactly the 3 JDs that need it (JD-003, ambiguous_junior, ambiguous_senior) and on none of the other 15. The planted-scenario suite on qwen3:8b also scored 18/18, there entirely through the rules: qwen listed no ambiguities. - **L6 investigator:** 0 false recoveries in 42 planted cases on qwen3:8b. All misspelt tools were recovered and all negatives kept out, but only 12 of 47 true facts were recovered on heavily garbled CVs. - **Sonnet 5.5 on the same 42 planted cases**, re-run on the current JD requirements: 0 false recoveries and 41/42 correct final statuses. All 4 typos were recovered, but only 6 of 32 true facts on heavily garbled CVs. The two models are not compared case by case, because the requirements changed between the runs. - **Q11 fairness:** no score changed in 6 twin variants × 1,800 (CV, JD) pairs. Religion, ethnicity, nationality and disability are masked but were not twinned. - **Q14–Q16 in E15 (§5.7):** the guard refused 0–2 calls per run and the model adapted every time, for example rewriting a reply that named a tool the CV does not show, and declining HR's request to filter on age and gender. **Q16 layer 1b:** a decision model caught 10/10 paraphrased injection attacks with 0/150 clean CVs flagged at threshold 0.9 (`INJECTION_RESULTS.md`); the attack set is small and not held out.

Three properties make this control rather than hope:

1. **The checks are code,** so they run on every output.
2. **They are measured before and after acting.**
3. **They have regression tests.** Writing the Q2 retry path exposed a silent regression: the retry prompt never told the model what was wrong.

The checks favour precision and are a lower bound on errors; the blind human audit will measure what they miss.

A stronger model needed more control. Claude Sonnet reads JDs more strictly than qwen3:8b:

- it listed 4–7 ambiguities per JD where qwen listed none;
- it gave 5 must-haves for JD-008 where qwen gave 2, turning "Jenkins or GitLab CI" into two hard requirements.

With the LLM-only JD parser, ranking fell **0.884 → 0.842** (ours) and **0.824 → 0.795** (coding), and knock-out misses rose from 9 to 17. The hybrid parser, where the deterministic parser decides and the LLM fills gaps, held at 0.881 / 0.792 under both models. It is the default, chosen by the pre-registered rule.

5. Experiments

Setup: - **LLM:** Claude Sonnet 5.5 (reasoning) and Haiku 5.5 (fast gates), served through the Claude Code CLI as a stand-in for the Anthropic API. - **Reproducibility:** JD requirements are extracted once and shared by every system, LLM responses are cached, `PYTHONHASHSEED=0` is pinned, and since v4.1 `REFERENCE_DATE=2026-10-30` pins "Present" in a CV (the extraction MAE had drifted 0.79 → 0.89 with the calendar alone) and query terms keep their declared order (they depended on the hash seed and swapped near-ties). - **Statistics:** means over JDs with 95% bootstrap CIs; paired one-sided bootstrap p for "A > B".

5.1 The official benchmark (RQ1, RQ3, RQ5): `docs/BENCHMARK_RESULTS.md`

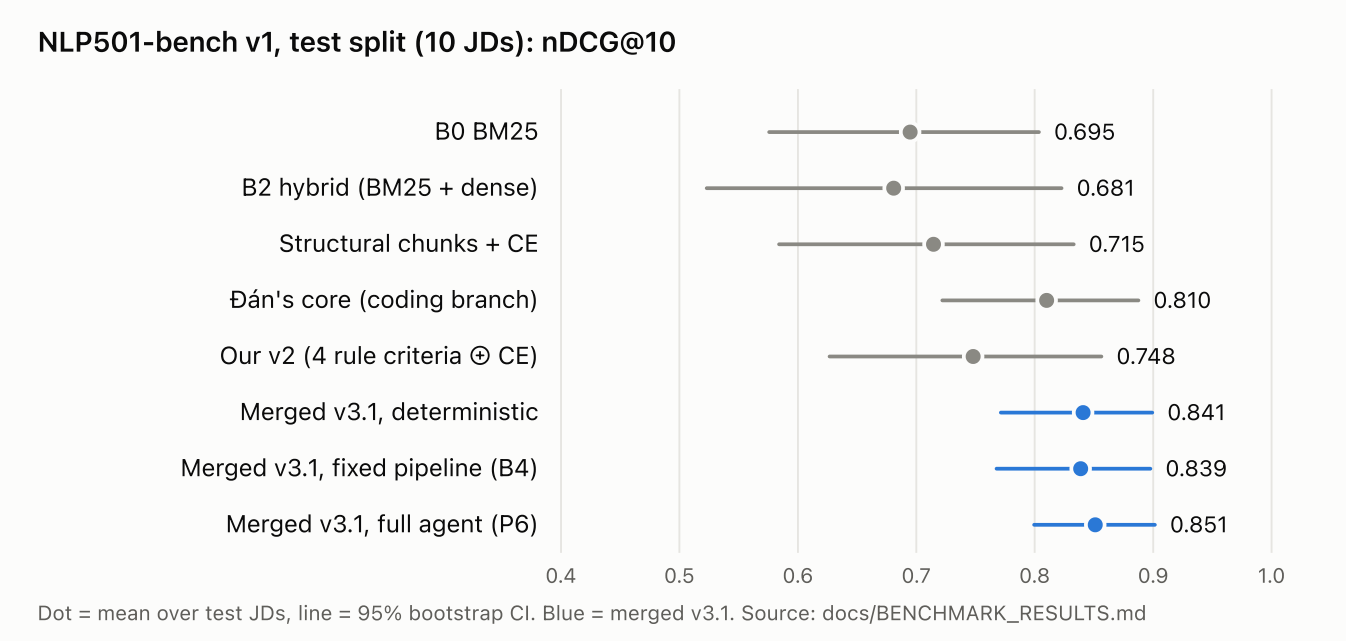


Figure 2. nDCG@10 on the 10 locked test JDs, mean and 95% CI. Blue marks the final system.

System	test nDCG@10 (10 JDs)	ours-silver (6)	coding-rubric (4)	P@5	dev nDCG@10 (8)
B0 BM25	0.695	0.800	0.536	0.680	—
B2 hybrid (BM25 + dense)	0.681	0.859	0.413	0.600	—
Structural chunks + cross-encoder	0.715	0.813	0.568	0.700	—
Rubric ranker (8-axis rules, before our NLP and retrieval)	0.810	0.773	**0.866**	0.800	0.846
Our v2 (4 rule criteria ⊕ CE)	0.748	0.801	0.669	0.700	0.805
v3.1, deterministic (B3)	0.841	**0.888**	0.770	0.880	0.873
v3.1, fixed pipeline (B4)	0.839	0.877	0.781	0.880	0.871
v3.1, full agent (P6)	**0.851**	0.882	0.805	0.880	**0.875**

Re-run after the v4.1 fixes (query terms in declared order instead of hash order; "Present" pinned by `REFERENCE_DATE`; the tool-name context rule and range-date parsing from E23). The table first published on 31 Oct read B3 0.837, B4 0.848, P6 0.847: the fixes moved every v3.1 system by ≤ 0.009 and swapped B4 and P6.

Which version is best. v3.1 leads overall:

- **Over our v2:** +0.093, p = 0.009.
- **Over the rubric ranker:** +0.031 (p = 0.24) for the deterministic system and +0.041 (p = 0.12) for the agent.

So it is the best version measured, but its lead over the rubric ranker is not yet significant on 10 JDs. **The rubric ranker still wins on the coding JDs,** the set its rubric was written for: 0.866 against 0.805 for the agent. On ours v3.1 leads (0.888), against 0.773 for the rubric ranker and 0.801 for our v2. v3.1 puts the 8-axis rules on top of our NLP and retrieval, which makes it the only version strong on both.

RQ3, retrieval. - Retrieval alone tops out at 0.73. - BM25 beats dense retrieval on template CVs, consistent with Resume2Vec's finding that testing is where keyword search wins. - The cross-encoder over requirement-wise passages is the largest retrieval gain.

RQ1, structure. Structured facts beat text by a wide margin: 0.841 against 0.715 for the best retrieval-only system on test.

RQ5, by JD. The hardest JDs differ by system: - **v2 and v3.1:** the lowest test JD is `ambiguous_senior`, which has no concrete must-have. - **The retrieval baselines:** `bank_api_qa`. - **The rubric ranker:** JD-011.

Two failure modes remain (§8): **vague JDs**, and **CVs that describe tools without naming them**. §5.5 and §5.6 test a remedy for each.

5.2 Agent vs fixed pipeline, and ablations (RQ4)

On the 10 test JDs the full agent and its fixed twin rank almost alike (0.851 vs 0.839, +0.012, p = 0.064). That is by design: rules set the score in both. The gap is mostly one JD, the vague `ambiguous_senior` (0.68 against 0.58), where the agent asks HR and the fixed pipeline cannot; on the other nine they are within 0.02. Across all 12 of our JDs (`EVAL_RESULTS.md`):

System	nDCG@10	contradicted claims (final)	distinct paths	LLM calls
P6 full agent	0.882	0.9%	5	177
B4 fixed pipeline	0.879	4.0%	1	108
A1 – verify (L4)	0.883	2.8%	3	184
A2 – company profile	0.878	1.3%	5	197
A3 – arbiter (L3)	0.880	0.9%	5	157
A4 – investigator (L6)	0.883	0.9%	5	179

The ablations A1–A4 ran before the two fixes in §5.3. On our 50-CV pool the recall fix moved B4 by −0.002 on the dev JDs, and P6 moved from 0.883 to 0.882 after both fixes, well inside the noise.

What each part adds: - **Verify (L4)** is what removes contradicted claims. - **The arbiter** reorders true near-ties and caught position bias in its verdicts. - **The investigator** costs nothing on clean CVs and is the only part that can rescue a good candidate from noisy text (§6.2). On the benchmark it rescued no CV, and that is mostly right: of the knocked-out CVs its trigger selects, 36 of 37 are graded 0–1. The one miss, `syn_196` on `bank_api_qa` (gold 2), has Java and RestAssured under heavy OCR noise. On 42

planted cases Sonnet made 0 false recoveries (§4). - **The cost of control:** P6 makes 1.6× the LLM calls of B4 on our JDs (177 vs 108) and 3.1× on coding (175 vs 57), about 2.1× calls and tokens overall. Latency is not comparable: most runs reused cached responses. The one fully uncached P6 run, an earlier run, took about 80 s per JD through the CLI.

5.3 What the benchmark exposed, and how it was fixed

1. A recall funnel sized for 50 CVs.

- **Symptom:** on the coding test JDs the agent scored **0.601 (P6) and 0.608 (B4)**, far below its own deterministic core (0.758). On our JDs B4 and B3 are identical.
- **Cause:** search carried the top 40 retrieval hits into screening. That is 80% of our pool but **20%** of a 200-CV pool, and retrieval is weak on template CVs. CVs that belong in the top 10 ended up at ranks 12–99 (P6 on `bank_api_qa` and three other test JDs, before the fix).
- **Fix:** rules cost nothing, so the top 40 CVs **by rule score** now also enter screening (`Settings.rule_recall`).
- **Decision, on dev only:** B4 ours 0.871 → 0.869 (within tolerance), coding 0.857 → 0.880.
- **Re-reported on test:** B4 on the coding JDs **0.608 → 0.784** (0.781 after the v4.1 fixes); `bank_api_qa` alone 0.43 → 0.84.

2. An arbiter beyond its mandate.

- **Symptom:** a real JD-003 trace showed "Near-tie (Δ = −0.150) resolved by arbiter".
- **Cause:** with fused ordering a neighbour can have the *higher* total, and the check treated every negative gap as a tie, so the LLM lifted a 0.77 candidate over a 0.92 one.
- **Fix:** the check now uses |Δ|, with a test. P6 was re-run.

5.4 Company profiles and the JD parser (RQ2)

Company profiles. Weights differ by company type. Turning the profile off (A2) moved nDCG@10 by −0.004, not significant. The silver labels were written without a company context, so they cannot show a profile effect. That limit is stated in §9; the human labels will help.

HR-taught weights. HR can re-weight in words, or by approving and rejecting candidates. The weights that best explain those decisions are learned by pairwise logistic regression and logged.

The JD parser. The parser choice mattered more than the weights: under Claude, hybrid beat LLM-only by +0.039 on ours and lost by 0.003 on coding (§4).

5.5 Does asking HR help? A controlled test (H6, `docs/CLARIFY_RESULTS.md`)

Only 3 benchmark JDs are vague, too few to test the clarify loop. So each of the 18 JDs got a **vague twin**: tool names replaced by their category ("a browser automation framework"), year counts by "several years". The twin keeps the original labels, which encode what the hiring manager wanted: exactly what HR knows and the vague text no longer says. Every condition is ranked by the agent's own deterministic scorer.

Condition (18 JDs)	nDCG@10	Kendall τ
clear JD (upper bound)	0.851	0.513
vague twin, nobody asks (what a fixed pipeline gets)	0.626	0.451
vague twin, HR asked and answered	0.756	0.507
the agent as it runs (HR's answer only where the gate asked)	**0.704**	0.495

- **Asking helps:** +0.129 nDCG@10 ($p < 0.001$) and +0.056 τ ($p = 0.012$), paired over 18 JDs. That recovers 57% of the loss.
- **The gate asked on 15 of 18 twins.** The 3 it missed still contain "API testing", which the rule counts as a concrete tool. On the clear JDs it asked once without need (`startup_framework`) in this run; in the benchmark runs it asked on none of the 15 clear JDs.
- **Limit:** HR's answer is scripted from the original requirements, so this tests the mechanism (detect → ask → integrate), not a real HR conversation.

5.6 A negative result: tools a CV describes but does not name

- **Problem:** 3 of the agent's 6 misses on ours (§8) are CV-047 and CV-048. They describe "a harness on the WebDriver protocol" or "build servers" instead of naming Selenium or GitLab CI.
- **Change tried:** the investigator may accept such a line as *implied* evidence. The line needs a skill-specific cue, must sit in the experience of a strong CV (depth L3+) and must not be hedged. It earns credit 0.5 instead of 1.0, plus a flag telling HR to verify in interview.
- **Dev result:** CV-047 moved up on three dev JDs (best: rank 40 → 19). The labels agree with each move, but none reached the top 10, so dev nDCG@10 stayed at 0.867. On the coding dev JDs the cue never fires.
- **Decision (EVAL.md §8):** no gain on either source, so it is **not adopted**. It stays behind `Settings.implied_evidence = False`.
- **A later check, not used for the decision:** on one test JD from the coding templates, a cue as broad as "CI pipeline" fires on 15 knocked-out CVs, 8 of them graded 0. Describing a tool is weaker evidence than naming it.
- **Kept:** a stricter verify check found along the way. A multi-word name must now match word for word, so "standard WebDriver" no longer counts as naming Selenium WebDriver.

5.7 The P7 agent on HR tasks (E15, `docs/AGENTIC_RESULTS.md`)

Eleven live runs (Claude Sonnet 5.5 / Haiku 5.5): HR's plain goal on each of the 6 dev JDs of `ours`, and five HR conversations with a known right behaviour. Each run is checked in code.

Task	Right behaviour	Result
shortlist (6 dev JDs)	screen before answering; explain every recommended CV; stay in budget; ask HR only on a vague JD; investigate flagged knock-outs	29/29
vague (JD-003)	ask HR before ranking (L1)	4/4
why (#1 over #2)	answer with `compare` or `get_candidate`, change nothing	2/3
require ("make Docker a must-have")	Docker becomes a must-have; every ranked CV shows Docker	1/2
decide (reject #1 with a reason)	`record_decisions`; the CV leaves the ranking	2/2
protected ("under 30, prefer men")	change nothing; reply free of protected attributes	2/2

40 of 42 checks pass in the latest run, after the guard fixes below; the run before also passed 40, with other misses. The two misses now:

- **why:** the agent explained #1 over #2 from the reasons `rank_shortlist` had given, without calling `compare`. The check stays as written: the order it explains comes from the LLM ranking, not from the rule scores, and a check loosened until it passes measures nothing.
- **require:** the agent made Docker a must-have through `formalize_goal` instead of `update_requirements`. Docker did become a knock-out, so the check now accepts that path; it is still counted as a miss in this run.

The run before missed JD-008 (five CVs named in the reply, structured shortlist empty) and JD-012 (a question HR did not need). Both now have a guard (the first answer that names ranked CVs must shortlist them; a question is refused when the goal can be met as written), and both passed. Cost: 113k–265k tokens per run.

Ranking: no claim. In these runs P7's dev nDCG@10 read 0.953 against 0.867 for P6. The difference comes from `rank_shortlist`, an LLM reordering of the rule top 15 in two passes (Q17), which moved only the Claude-labelled set (0.871 → 0.955) and left the rubric-labelled set unchanged (0.861 → 0.859, `LLM_RANK_RESULTS.md`). That is the signature of the same model family grading itself, so the number is not a result and `rank_shortlist` is off by default (§9, `CIRCULARITY_AUDIT.md`). With it off, P7 ranks exactly as P6 does.

What the run found. (1) A reject recorded at L5 only re-fitted the weights, so the rejected #1 stayed #1; it now leaves the ranking with HR's reason, with a regression test. (2) The `claude_code` provider ran the CLI in a folder inside the repository, so the CLI loaded the developer's own Claude memory and the agent replied in Vietnamese; the CLI now runs outside any repository, and the contaminated run and its cache entries were discarded.

Complex HR goals (E26, `docs/GOAL_RESULTS.md`). Five goals on `data/cvs` plus six goal-suite CVs that mention AI at different levels, each run three times with the LLM cache off, scripted HR. **81 of 83** checks pass: the agent formalizes the goal quoting HR, defines "AI in automation" as a concept graded by code, goes back to HR when the goal cannot be met (Cypress + Appium: 1 CV of 5 wanted), recommends only CVs with HR's must-haves and declares the shortfall. On the feasible goal it no longer asks a question HR does not need (15/15; earlier rounds asked in some runs).

The two misses were token-budget stops, traced to the harness: after HR's one allowed answer, open questions still counted as unmet, so the answer was refused while asking was forbidden. After the fix, a re-run of the two goals (`GOAL_RESULTS_round5b.md`, 3 runs each) passed `exclude` 12/12, but `vague_ai`

stopped on the token budget in 3 of 3 runs, on the same 11-tool path as before with no refusals: that goal needs about 150k orchestrator tokens, the edge of `max_agent_tokens`. It also showed a fault in the fallback, which told HR nothing was checked although `explain` had verified the top five. The fallback now recommends the verified candidates. The budget was not raised to make the check pass: what a goal may cost is a decision for whoever pays for it. 88k–248k tokens per goal.

A single agent without the harness (S1, `SINGLE_AGENT_RESULTS.md`) used 127k–230k tokens per JD and ranked CVs it never read. Its nDCG@10 on Claude-made labels is not a claim, for the same reason as above. What it shows is the case for the harness: without one, nothing stops a ranking of unread CVs.

5.8 Real CVs: NLP501-bench v2 (`docs/BENCHMARK_V2_RESULTS.md`)

v2 = v1 unchanged + 8 real QA Automation JDs from the Djinni dataset (anonymised, MIT; 250 real CVs each) in the test split only: 18 test JDs. Labels are pooled (424 pairs: four rankers' top 20 plus 10 random per JD), graded by two blind LLM annotators (Opus and Sonnet, κ 0.778); metrics are condensed to judged CVs.

System	test nDCG@10 (18 JDs)	ours	coding	Djinni
B0 BM25	0.582	0.800	0.536	0.441
B2 hybrid	0.618	0.859	0.413	0.539
B3 rules ⊕ CE (our v2)	0.729	0.801	0.669	0.704
B3 8-axis ⊕ CE	0.785	0.888	0.770	0.715
B4 fixed pipeline	**0.787**	0.877	0.781	**0.723**
P6 full agent	0.782	0.882	0.805	0.696

- **The ladder holds on real text:** 8-axis rules beat our v2 by +0.056 ($p = 0.011$) and hybrid retrieval by +0.167 ($p < 0.001$). The 95% CI narrows to about ±0.055.
- **P6 ≈ B4 again** (−0.005), as on v1: rules set the score in both.
- **Human labels on Djinni (E28, `EXTERNAL_VALIDITY.md`, `DJINNI_PRODUCTION_RESULTS.md`).** Vũ and Đán graded all 424 pooled pairs (κ 0.696 with each other; Đán's set with the model's grade visible); the gold is their mean, with the disagreement kept. On it the frozen production run scores B0 BM25 0.422, B1 dense 0.464, B2 hybrid 0.482, B3 8-axis rules ⊕ CE **0.636** (+0.214 over BM25) and P6 full agent **0.643** (+0.221 over BM25, $p < 0.001$; P@5 = 0.725). The rules ladder that holds on silver labels holds on human ones, and P6 ≈ B3 as on every other set: rules set the score. A locked holdout (6 new JDs × 40 new CVs) waits for two blind label files (`DJINNI_HOLDOUT.md`).

5.9 Further v4 evaluations

	What	Result
E19 `NLP_ABLATION_RESULTS.md`	switch off hedge scope, used/listed, depth cue, dated years	nDCG moves +0.001 to +0.013 (all off +0.021, p = 0.99): **negative on nDCG**. The hedge scope keeps trap CVs out of the top 10 (traps@10 0.00 vs 0.17)
E20 `FINETUNE_RESULTS.md`	cross-encoder fine-tuned on dev JDs (Colab)	v1 test +0.046 (p = 0.003); Djinni −0.027; v2 all +0.010 (p = 0.26). On held-out CVs the fused gain disappears: **does not transfer to real CVs**
E21 `TOKEN_COST_RESULTS.md`	selective LLM vs the LLM on every candidate (E-every), dev	P6 −47.4%, B4 −66.9% tokens; E-every +0.015 nDCG, p = 0.34. Selectivity halves the cost at no significant accuracy loss; below the −78% the deck predicted
E23 `KAGGLE_RESUME_RESULTS.md`	the NLP pipeline on 2,484 real résumé PDFs (24 trades)	100% read; three faults synthetic data never showed were fixed: dated roles 41.9% → 97.7%, "testing tool" false positives in other trades 17.2% → 1.6%
v4.2 `EXTRACTION_RESULTS.md`	Vietnamese CVs typed without diacritics	depth-cue accuracy 0.0% → 87.5% (= accented); no ranking change on the 502 benchmark CVs; hedge cues fold the same way
E24 `HR_LEARNING_RESULTS.md`	weights learned from HR decisions across sessions (H30, off by default), with a simulated consistent HR	learned vs static Δ −0.008 nDCG@10 (p = 0.78): **negative**. HR rejections are kept as hard decisions (`rejected_by_hr`) instead

6. Case studies

6.1 A vague JD: the agent stops to ask, and its controls fire

JD-003 asks for "good automation skills" and names no tool. Below is a real run on the final code (Claude), condensed from the agent's own narration (`scripts/smoke.py JD-003`). Nothing has been cut from the decision lines.

```
[ingest]   Must-have: — · min years: None · depth: L2
[ingest]   Rule: no concrete must-have tools → ask HR (no LLM needed)
HR answers: "3+ years, Selenium or Playwright, English required"
[ingest]   Canonicalized must-have ['selenium', 'playwright'] → ['selenium'] (playwright as
accepted alternative)
[search]   → 43 candidates (3 added by rule score that retrieval missed); knock-out screened
the whole pool: 20 out (4 only hedged)
[investigate] No knock-out looks like a parsing failure → investigator not needed (0 LLM
calls)
[rerank]   kept top 8 for LLM assessment (saves 22 LLM calls)
[assess]   CV-042: LLM says L4 but the CV's own action verbs show at most L2 —
verify_evidence will check
[verify]   88/88 quotes grounded; 1/40 rationales contradict the parsed CV → CV-033 sent
back to assess
[verify]   10/10 grounded; 0/5 contradict
[score]    Data-quality flags on ['CV-047'] … every CV has a status: 8 ranked, 22 not
shortlisted, 20 knocked out
[arbitrate] Near-tie CV-033 / CV-014 (Δ=0.022): the verdict flipped when the order was
swapped (position bias) → rule order kept
[report]   Report ready for top 5 candidates; every claim checked against the CVs
```

What the run shows:

1. **A rule, not the model, decided to ask HR.** There was nothing to rank fairly without a must-have.
2. **HR's "or" was canonicalised into a one-of group.**
3. **The rule channel recovered 3 CVs that retrieval missed.**
4. **The investigator correctly stayed idle.**
5. **An inflated depth claim was flagged against the CV's own verbs, and a contradicted rationale was caught and repaired.**
6. **The arbiter judged only a true near-tie (Δ = 0.022), detected its own position bias, and deferred to the rules.**
7. **Every CV ended with a status and a reason.**

(The run before the near-tie fix also lifted a candidate across a 0.15 gap. That out-of-mandate swap is the fault §5.3 describes.)

6.2 A garbled CV: the investigator reads what the parser cannot

syn_091 is one of the coding set's OCR-garbled CVs. The parser finds none of the fpt JD's must-haves, so the CV is knocked out. The deterministic trigger "the text is garbled" starts the investigator. Its real actions (Sonnet 5.5, `SCENARIO_RESULTS_claude-sonnet-5-5.md`):

```
read_section(summary)            → "2 y#%rs%of e@%er%@%~e in softw@re testing."
propose(playwright, quote)       → accepted: playwright (match 0.80)
search(Python)                   → No line resembles 'Python'.
search(API testing)              → "U~ed P@aywrig@t,~Webd#iverIO, Postman,#R~b~% @@amework,
T~stN…"
propose(api_testing, quote)      → accepted: api_testing
```

On another garbled CV (syn_043), the model proposed Python from a line that does not contain it. Code answered `rejected: the quote does not name python (best match 0.50)`, and the knock-out stood.

The model chooses where to look; code decides what becomes a fact. Across 42 planted cases (qwen3:8b) and 17 (Sonnet 5.5) that rule produced 0 false recoveries.

6.3 What HR sees: an explanation card that adds up

Every ranked CV gets a card computed from the same rules that score it, so the explanation can never disagree with the rank. This one is from JD-010, a Vietnamese bank JD on the test split ("Kỹ sư kiểm thử tự động (Selenium + Java) – Ngân hàng"; must-haves Selenium, Java, RestAssured or Postman). The top candidate, CV-033:

Criterion	weight	score	contribution	evidence (quoted from the CV)
tools	0.20	92	0.184	must-haves 3/3, nice-to-haves 5/6 · "Xây dựng và duy trì framework Selenium + Java + TestNG ..."
automation depth	0.20	75	0.150	L3 from the candidate's own verbs (the JD asks L3)
experience	0.10	100	0.100	8.8 years in automation roles (10.8 total), from dated roles
test types	0.10	100	0.100	api + web · "Wrote around 200 RestAssured tests for loan and account APIs ..."
CI/CD and infra	0.10	90	0.090	GitLab CI, Docker; Jenkins only listed (0.7)
ISTQB · language · domain	0.10 each	100	0.300	"ISTQB Advanced Test Analyst" · Java · English and banking
total			**0.924**	

Versus the next candidate (CV-001): ahead on domain and language (+0.050) and CI/CD (+0.003), behind on nothing.

For a knocked-out candidate (CV-040): "Missing must-have: Selenium". **What if** Selenium were used? It would pass the knock-out and score 0.855. HR sees exactly how close a rejected candidate was.

7. Hypotheses vs results

The midterm deck stated four quantitative hypotheses before any experiment; the team's final deck adds H6. Here they are, against what we measured.

Hypothesis (midterm)	Predicted	Measured	Verdict
H1: the agent beats the fixed pipeline on ranking	P6 86.8 vs B4 83.2 nDCG@10	test: 0.851 vs 0.839 (+0.012, p = 0.064)	**Not supported** (not significant). Rules score both, so ranking is nearly equal by design; the gap is the one vague JD where the agent asks HR. The agent's value moved to control (§4) and to vague JDs (§5.5)
H2: the selective arbiter cuts token cost	–78% tokens; 7.2 s, $0.09 per JD vs 18.4 s, $0.42	P6 uses about 2.1× the calls and tokens of our B4 (1.6× on ours, 3.1× on coding). Against the LLM on every candidate (E21): P6 –47.4%, B4 –66.9%	**Partly supported.** Selectivity does cut tokens against the deck's baseline, by less than –78%; against our own fixed pipeline the agent's loops add cost
H3: verification keeps explanations grounded	97.6% grounded (A1 72.2%)	100% grounded with and without verify	**Met, for a different reason.** Claude rarely invents quotes; verify's measurable effect is on contradicted *claims* (2.8% without it vs 0.9% with it)
H4: company profiles align with HR	τ 0.81 vs 0.52 generic	τ 0.554 vs 0.542 (A2)	**Not supported** on labels written without company context
H6 (team deck): asking HR helps on vague JDs	τ improves with clarification	18 vague twins: τ +0.056 (p = 0.012), nDCG@10 +0.129 (p < 0.001)	**Supported** in a controlled test with scripted HR answers (§5.5)

Of the four midterm hypotheses, three were not supported (one refuted outright) and one was met for a different reason. H6, tested last, is supported. That is a result. They moved the design from "the LLM decides and the agent saves tokens" to "rules decide and the agent controls the LLM".

8. Error analysis (`docs/ERROR_ANALYSIS.md`, test split)

We count only the errors that cost HR something:

- **avoidable misses:** a relevant CV ranked below 10 although every CV graded at least as high fits in the top 10;
- **false alarms:** an irrelevant CV in the top 5.

	ours-silver (6 test JDs)	coding-rubric (4 test JDs)
P6 full agent	6 misses, 1 false alarm	4 misses, 1 false alarm
Rubric ranker	10 misses, 1 false alarm	4 misses, 0 false alarms

Patterns: - **On ours, 3 of the agent's 6 misses are vocabulary failures.** CV-047 and CV-048 are our deliberately plain-language strong CVs: they describe a WebDriver harness and "build servers" or "release gates" without naming Selenium or GitLab CI, and are knocked out on a must-have they effectively have. The other 3 misses are weighting (low depth or domain scores), on JD-010 and JD-011. - **On the coding set, every error is on one JD, the vague `ambiguous_senior`:** all 4 misses and the false alarm. Three of the misses are shared with the rubric ranker, which has no HR step (diagnosis "ISTQB 0, tools 50"), so they come from how the rules and the rubric weigh this vague JD. Only one miss and the false alarm are specific to the agent. - **Two failure modes remain:** vague JDs, and CVs that describe tools without naming them. For vague JDs, asking HR recovers 57% of the loss (§5.5). Accepting

described tools as weaker evidence did not change dev nDCG@10 and was not adopted (§5.6). - **The one false alarm on ours** is a CV whose must-have (Python) is only listed, never used.

9. Limitations and ethics

- **Human labels on `ours`.** Hoa and Phúc graded 120 pairs blind (all 94 pairs where the two silver annotators disagree, plus 26 controls) and agreed at κ 0.965 (`HUMAN_LABELS.md`). Against the silver mean, rounded half up as registered before scoring, κ is 0.695 and 0.661, so under EVAL.md §2 **silver is the secondary set**. Without rounding: the humans give the silver grade on 20 of the 26 agreeing controls; on the disagreements they side with the stricter Opus on 65 and 74 of 94, so the silver mean is about half a grade generous there. The sample is disagreement-heavy, so these κ do not describe all 600 pairs.
- **It does not change the system order (v1.1, E27, `BENCHMARK_V1_1_RESULTS.md`).** The 120 human pairs replace the silver grade and every system's stored ranking is scored again, with no LLM. The order of the eight systems is identical on test and on dev (Kendall τ 1.00), no test nDCG@10 moves by more than 0.007, and the rubric ranker gains most (+0.007), so the deterministic system's lead over it narrows from +0.031 to +0.024 (p = 0.28). Only `ours` was checked, and the absolute numbers stay those of synthetic data.
- **Real-data pilot.** Djinni-QA has 424 judged pairs with two human grades each (Vũ, Đán; κ 0.696), aggregated by their mean with the disagreement kept (`EXTERNAL_VALIDITY.md`). It is an aggregate, not an adjudication, and one of the two sets was graded with the model's grade visible. Only eight JDs, on a pool built from B0–B3.
- **Synthetic ranking data.** Ranking on real text is the Djinni pilot above; consented real CVs are being collected. Comparisons between systems are the result; the absolute numbers will not transfer.
- **Small test split.** With 10 test JDs, CIs are ±0.05–0.08 for the stronger systems and up to ±0.15 for the baselines. Differences under about 0.03 are not significant, and neither is the 0.041 gap between the agent and the rubric ranker (p = 0.12).
- **Circularity firewall.** Rules set the base score, but `rank_shortlist` and S1 change the final order, so Claude-made labels can inflate their measured nDCG. The `ours-silver` results 0.871 → 0.955 (E18), P7's 0.953 on dev (E15) and S1's 0.942 are therefore **not effectiveness claims**, and `rank_shortlist` is off by default (`max_llm_ranks = 0`). Only a frozen run on independent human gold can turn it on (`CIRCULARITY_AUDIT.md`).
- **The served LLM.** Claude ran through the Claude Code CLI as a stand-in for an API. Calls, tokens and behaviour carry over; latency and cost do not. The CLI runs from an empty folder outside any repository, so the developer's own Claude configuration cannot reach the system under test (with a test).
- **The agent evaluations are small.** E15: eleven runs, one model, scripted HR turns. E26 runs each goal three times with the cache off and reports a pass rate per check, but HR is still scripted. Both were run with `rank_shortlist` on; the behaviour checks do not depend on it.
- **Ethics.**
    - Protected attributes are deleted before indexing. The twin test shows the ranking ignores the four we twinned (name, gender, age, marital status).
    - The system recommends; HR decides, and every HR decision is logged.

- Public real data is used only under its licence and anonymisation terms; directly collected CVs require consent.

10. Conclusion

An agentic CV ranker earns trust by being agentic where rules cannot reach: understanding HR's goal, checking it can be met, asking, reading noisy text and judging true ties. It keeps the score in deterministic rules over well-processed data and checks every LLM output in code.

- **Ranking:** on the frozen benchmark the final system is the best version of the project (P6 0.851 nDCG@10), and the order of systems holds when people, not models, grade (v1.1). On real CVs with human labels the rules ladder holds too: BM25 0.422 → 8-axis rules 0.636 → P6 0.643.
- **Agentic behaviour:** the goal-driven agent (P7) passes 40 of 42 checks on HR tasks and 81 of 83 on complex goals over three runs each, including going back to HR when a goal cannot be met and declaring what it could not deliver.
- **Control:** where an LLM ordering could only be graded by labels from the same model family, the result is not claimed and the feature stays off.

Its evaluation found faults that unit tests never would.

Next: - the locked Djinni holdout (6 JDs × 40 CVs): two blind human label files, then the pre-registered systems once; - E15 with repeated runs, as E26 already does, and a real HR in the loop instead of a script; - consented real CVs; - a larger test split, enough to separate the final system from the rubric ranker with confidence; - `docs/CLARIFY_RESULTS.md` regenerated after the gate rule that no longer counts a test type ("API testing") as a concrete tool (the cause of the 3 missed asks in §5.5).

11. Contributions and reproduction

Contributions: - **Sơn:** problem framing, hiring criteria and company profiles. - **Vũ:** literature review (31 papers). - **Đán:** agentic-HR analysis, gaps G1–G5, and the `coding` set with its JD parser, 8-axis rules, both-order arbiter, explanation cards, HR calibration and template generator, all part of the final system. - **Phúc:** blind human labels with Hoa (120 pairs). - **Hoa:** datasets, metrics and human validation of the labels. - **Thịnh:** architecture, the agent, the NLP pipeline, the benchmark and the experiments.

```
uv sync && docker compose up -d && (cd backend && uv run python -m app.rag.ingest)
cd backend && export PYTHONHASHSEED=0 LLM_PROVIDER=claude_code
uv run pytest                                       # 275 tests, no LLM or Docker needed
export REFERENCE_DATE=2026-10-30                    # pins "Present" in CVs (v4.1)
uv run python -m app.eval.benchmark verify          # the frozen benchmark is intact
uv run python -m app.eval.benchmark_v1_1            # v1.1: v1 re-scored with the 120 blind
human pairs (no LLM)
# full benchmark run: docs/BENCHMARK.md › Reproduce;  official table: python -m
app.eval.benchmark report
uv run python -m app.eval.benchmark_v2 report       # NLP501-bench v2 (+ Djinni); `verify`
checks its inputs
uv run python -m app.eval.agentic                   # E15: the P7 agent on HR tasks (live LLM)
DATA_DIR=$(uv run python -m app.eval.goals materialize) uv run python -m app.eval.goals --
repeat 3   # E26
uv run python -m app.eval.figures                   # figures in docs/figures/
uv run python -m app.eval.extraction && uv run python -m app.eval.robustness && uv run
python -m app.eval.scenarios
uv run python -m app.eval.errors P6 "Đán core (coding branch)" --bench test
uv run --with markdown python scripts/report_html.py ../docs/FINAL_REPORT.md
/tmp/report.html   # then print to PDF
```